

Lesson Notes: Debugging in Scratch 3

1. Lesson Introduction

In programming, it is common for projects to **not work as expected**.

Debugging is the process of **finding and fixing errors (bugs)** in your code.

In Scratch, debugging ensures that scripts **run correctly**, sprites behave as intended, and interactive projects work smoothly.

2. Learning Objectives

By the end of the lesson, learners should be able to:

1. Explain what **debugging** is.
2. Identify common **types of errors** in Scratch.
3. Use **strategies** to find and fix errors.
4. Test and correct Scratch projects.
5. Explain the importance of debugging in programming.

3. Key Vocabulary

- **Debugging** – the process of finding and fixing errors in code.
- **Bug** – a problem in a program that prevents it from working correctly.
- **Script** – a set of blocks in Scratch that perform an action.
- **Test** – running your program to check if it works correctly.
- **Logic Error** – the program runs but does not do what is intended.
- **Syntax Error** – a mistake in the structure of the blocks (less common in Scratch).

4. Common Types of Errors in Scratch

Type of Error	Description	Example
Syntax Error	Mistake in block placement	Missing a loop or putting a block outside its required sequence
Runtime Error	Occurs when the program runs	Sprite moves off stage or variable becomes negative
Logic Error	Program runs but does the wrong thing	Score does not increase correctly, or sprite moves in wrong direction

5. Debugging Strategies

1. Test Frequently

- Run scripts often to check if the code works as expected.
- Use the **green flag** to start and observe scripts.

2. Check Block Placement

- Ensure blocks are connected correctly.
- Check loops, if statements, and broadcast triggers.

3. Use Say / Print Statements

- Use `say` blocks to display **variable values** or messages at different points.
- Helps identify **where the problem occurs**.

4. Check Variables and Conditions

- Verify that variables are initialized correctly.
- Check if conditions in **if** / **if-else** blocks are correct.

5. Simplify the Problem

- Break your program into **smaller parts**.
- Test each part separately before combining scripts.

6. Ask for Help / Compare

- Compare your code with **examples or instructions**.
- Ask peers or teachers if you cannot find the error.

6. Teacher Demonstration (Step-by-Step)

Example: Debugging a Moving Sprite

Problem: The sprite does not stop when touching an obstacle.

1. **Check scripts:** Ensure there is an **if statement** to detect touching the obstacle.

```
forever
if <touching Obstacle?> then
say "Ouch!"
stop [all]
```

2. **Test the program** and observe behavior.
3. **Use say block** to print variable values or messages at each step.
4. Fix the block connections or variable errors.
5. Test again until the sprite behaves correctly.

7. Guided Practice for Learners

1. Give learners a Scratch project with **errors** (e.g., sprite moves incorrectly, score not updating).
2. Ask learners to:
 - Identify the **error**.
 - Explain why it is happening.
 - Correct the script.
 - Test the program to ensure it works.

8. Independent Practice

Learners should:

- Create a small Scratch project (e.g., a clicker game or sprite movement).
- Intentionally add a **bug** (e.g., wrong variable change, missing block).
- Swap projects with a peer and **debug each other's project**.
- Record the **bug found and how it was fixed**.

9. Common Mistakes

- Not testing scripts after making changes.
- Ignoring error messages or incorrect behavior.
- Forgetting to reset variables when starting a new run.
- Misplacing blocks in loops or conditional statements.

10. Assessment Questions

1. What is debugging?
2. What is a logic error in Scratch?
3. Name two strategies for debugging a project.
4. Why is it important to test your program frequently?
5. How can variables help in debugging?

11. Summary

- Debugging is an essential part of programming.
- Types of errors include **syntax, runtime, and logic errors**.
- Strategies include **testing, checking blocks, using say blocks, checking variables, and simplifying the problem**.
- Debugging ensures Scratch projects **work correctly and interactively**.